SOME REAL-WORLD STREAM CIPHERS

1

AN EXAMPLE: DVD ENCRYPTION

EXAMPLE: DVD ENCRYPTION (1)

- The **Content Scrambling System** (**CSS**) is used for protecting movies on DVD disks
- It uses the CSS stream cipher to encrypt movie contents using a **40-bit secret key**
  - CSS was introduced in 1996

- Ciphers using 40-bit keys are insecure because of brute force attacks
- The CSS stream cipher is particularly weak as it can be broken in far less time than an exhaustive search over all *2^(40)* seeds
  - This is well within the reach of currently available computational power
  - On a Pentium II (microprocessor commercialized in 1997), the attack takes only 1 minute

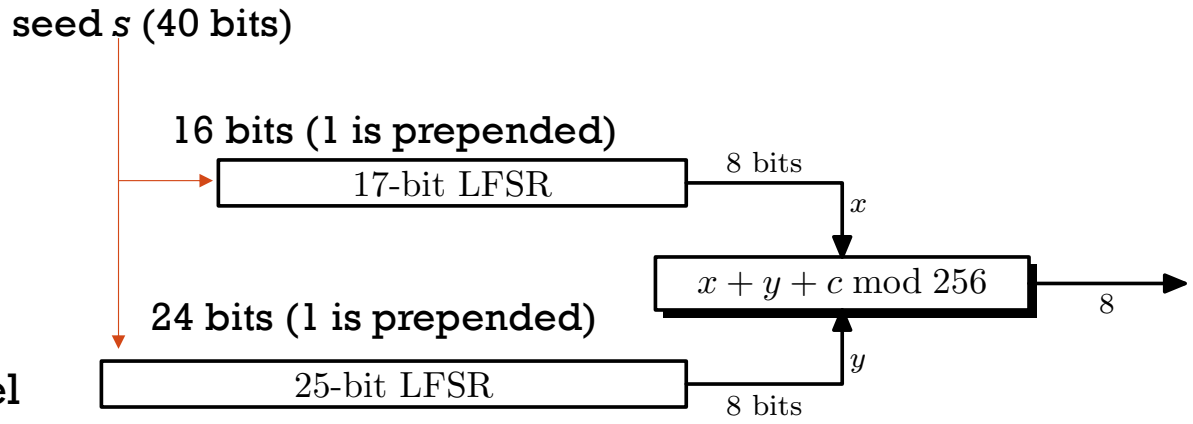
EXAMPLE: DVD ENCRYPTION (2)

- CSS uses LFSRs as hardware implementations are quite cheap
  - This makes LFSRs attractive for low-cost consumer electronics such as DVD players, cell phones, and Bluetooth devices
- To alleviate the problems deriving from the main weakness of LFSRs, the idea is to **run some LFSRs in parallel for some clock cycles and combine their results by means of some non-linear function**
  - The CSS stream cipher combines two LFSRs
  - The A5/1 stream cipher used to encrypt GSM cell phone traffic combines the outputs of three LFSRs
  - The Bluetooth E0 stream cipher combines four LFSRs
  - **All these algorithms have been shown to be insecure and should not be used**: recovering the plaintext takes far less time than an exhaustive search on the key space

EXAMPLE: DVD ENCRYPTION (3)

- Taps for the 17-bit LFSR
  - 14, 0
- Taps for the 25-bit LFSR
  - 12, 4, 3, 0
- The two LFSRs are run in parallel for 8 cycles and then the resulting bits are considered as integers and added modulo 256
  - *c* is the carry bit of the added

2^(40) attempts at most



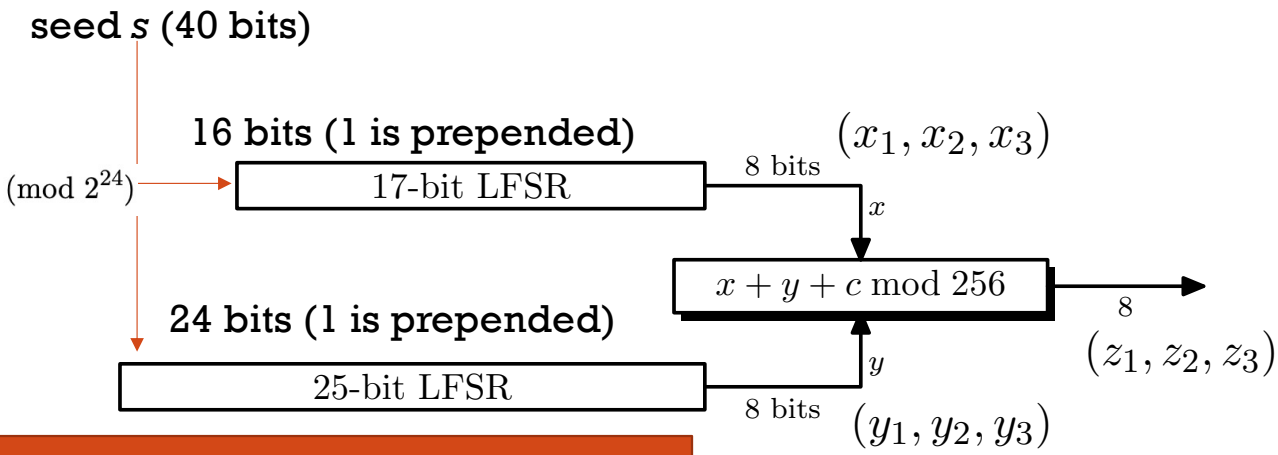
Attack (stupid brute force)

- Suppose Eve is given the first 100 bytes of the output sequence of additions
- Eve guesses a 40 bits seed
- Run for 100 iterations
- Compare results with those of the given sequence
- If equal, then guessed seed is correct, otherwise retry

EXAMPLE: DVD ENCRYPTION (4)

- Observation

$$(2^{16}x_3 + 2^8x_2 + x_1) + (2^{16}y_3 + 2^8y_2 + y_1) \equiv (2^{16}z_3 + 2^8z_2 + z_1) \pmod{2^{24}}$$



Attack (smart brute force)

- Suppose Eve is given the first 100 bytes of the output sequence of additions and of the output sequence of the 17-bit LFSR
- Eve guesses a 16 bits seed and compute the first 3 bytes (x_1, x_2, x_3)
- By using the observation above, Eve computes (y_1, y_2, y_3) and is able to determine the seed for the 25 bits LFSR
- Run for 100 iterations
- Compare results with those of the given sequence
- If equal, then guessed seed is correct, otherwise retry

2^(16) attempts at most

MORE INFO ABOUT CSS & WEAKNESSES

- https://en.wikipedia.org/wiki/Content_Scramble_System
- https://www.cs.cmu.edu/~dst/DeCSS/Kesden/index.html

7 A FAMILY OF CIPHERS: A5

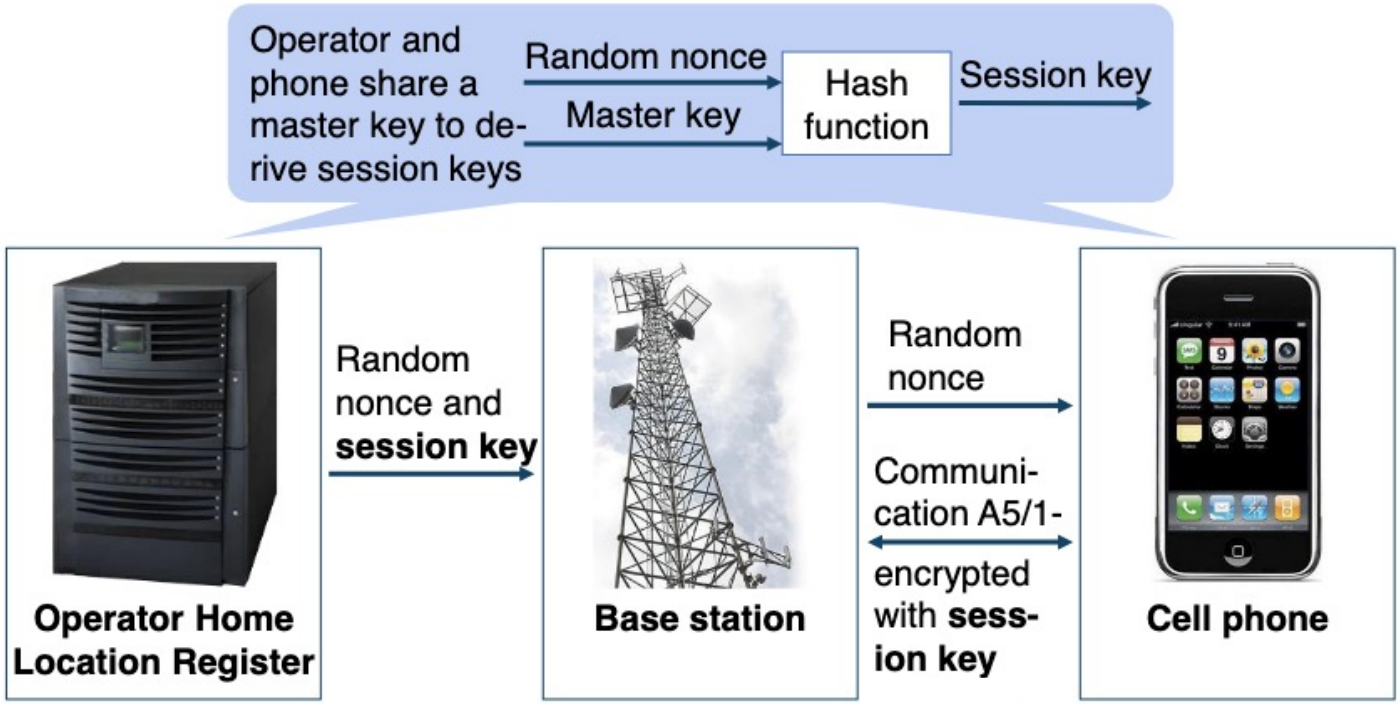
GSM

- ETSI = European Telecommunications Standards Institute (https://www.etsi.org/)
- Independent, not-for-profit, standardization organization

- GSM = Global System for Mobile communication (ETSI standard)
  - Protocols for 2G digital cellular mobile networks
  - GSM was the first near to global standard of wireless telecommunication
  - GSM provides services to more than 6 billion people worldwide (over 90 % market share)

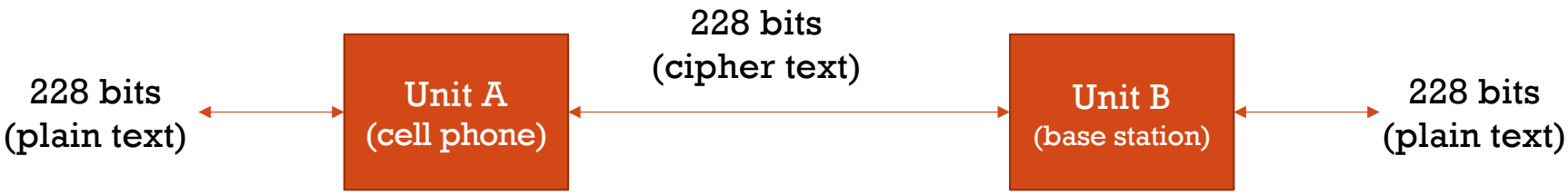
- When the GSM standard was designed from 1982-1991, the level of security specifications regarding both authentication and encryption were limited
  - Specific algorithms were never officially published (**security by obscurity?**)
  - Algorithms were **reverse-engineered** or **leaked**, leading to revelations of several possible attacks
  - Within a few months after the release, most of the cryptographic schemes had been compromised and some were even proven to be close to useless

GSM SETUP AND USE OF A5



AN INTRODUCTION TO A5 (1)

- GSM was originally designed for basic commodity phones and a security scheme could therefore be implemented on top of simple hardware components
- A5 was specified to encrypt over-the-air transmissions on the GSM network



- 1 frame = 228 bits
- 28 frames per second

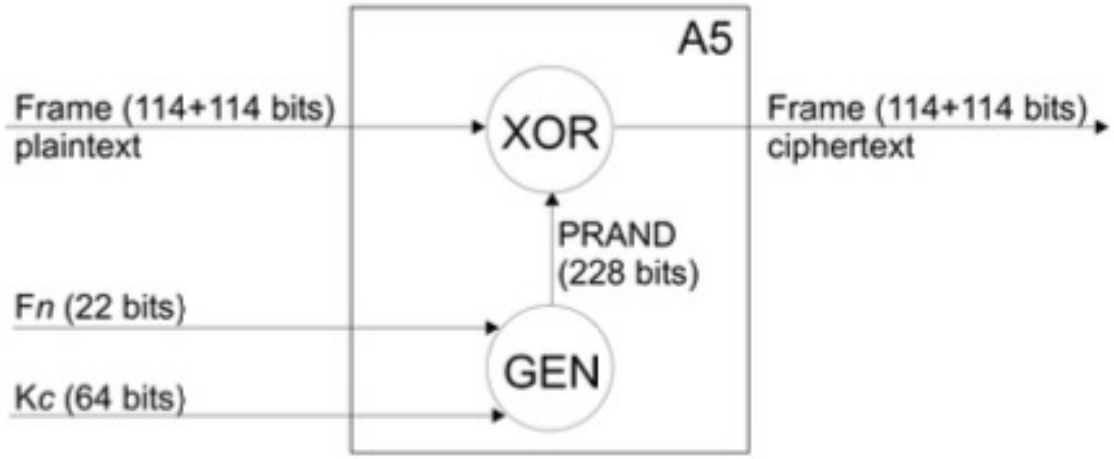
- 114 bits for **uplink** (mobile to base)
- 114 bits for **downlink** (base to mobile)

- Each frame is numbered by a **frame number**, obtained by a **frame counter** initialized with 0 at conversation-start and incremented by 1 (mod 2^22) with each frame sent

AN INTRODUCTION TO A5 (2)

- An algorithm of the A5 family **takes**
  - the session key *Kc* (symmetric) and
  - a frame counter *Fn*

  generates
  - 228 pseudo random bits (PRAND), called a key stream
- The key stream is then XORed with a 228 bit segment of plain text, yielding 228 bits of ciphertext



- Since encryption is very simple, what does make A5 secure?
- **Answer**: design/implementation of GEN
- Members of the A5 family differs on how GEN is implemented, thereby providing **different levels of security**

A5: SHORT OVERVIEW

- The GSM is de facto the standard protocol for 2G digital cellular networks used by mobile phones
- Data are transmitted in blocks of 228 bits
- First standard to introduce cryptographic algorithms for security
- It comes in 4 variants
  - A5/0: no encryption
  - **A5/1: stream cipher based on LFSRs**
  - **A5/2: stream cipher based on LFSRs**
  - A5/3: stream cipher based on the block cipher KASUMI

A5/0

- It is the weakest of the A5 versions as it **offers no encryption**
- It is a **no-operation cipher**, that generates the pseudo random bits by negating the input frame, thus leaving out the XOR function
- The result is an algorithm that outputs the plain text it received as an input

- This version is found in third world countries or countries with UN sanctions

14

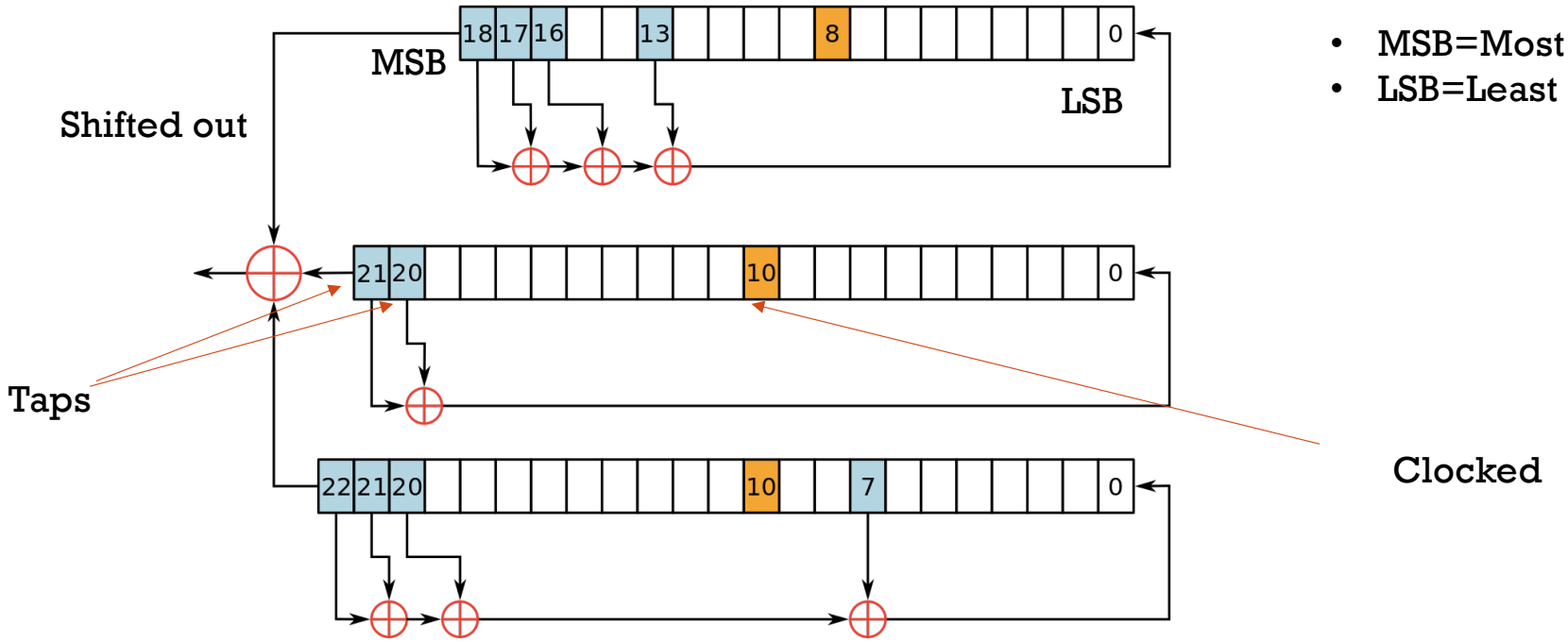
A5/1

A5/1: MAIN FEATURES

The three polynomials are primitives with periods:
- 2^(19)-1 = 524,287 (R1)
- 2^(22)-1 = 4,194,303 (R2)
- 2^(23)-1 = 8,388,607 (R3)

- year: 1987
- keystream length: 228 bits
- number of LFSRs: 3
- update of registers: majority function on the control bits
- Registers
  - lengths: 19, 22, 23 (total: 64 bits of state)
  - R1, R2, R3 are maximum-length LFSRs as the polynomials are primitive

LFSR number	Length in bits	Feedback polynomial	Clocking bit	Tapped bits
1	19	$x^{19} + x^{18} + x^{17} + x^{14} + 1$	8	13, 16, 17, 18
2	22	$x^{22} + x^{21} + 1$	10	20, 21
3	23	$x^{23} + x^{22} + x^{21} + x^{8} + 1$	10	7, 20, 21, 22

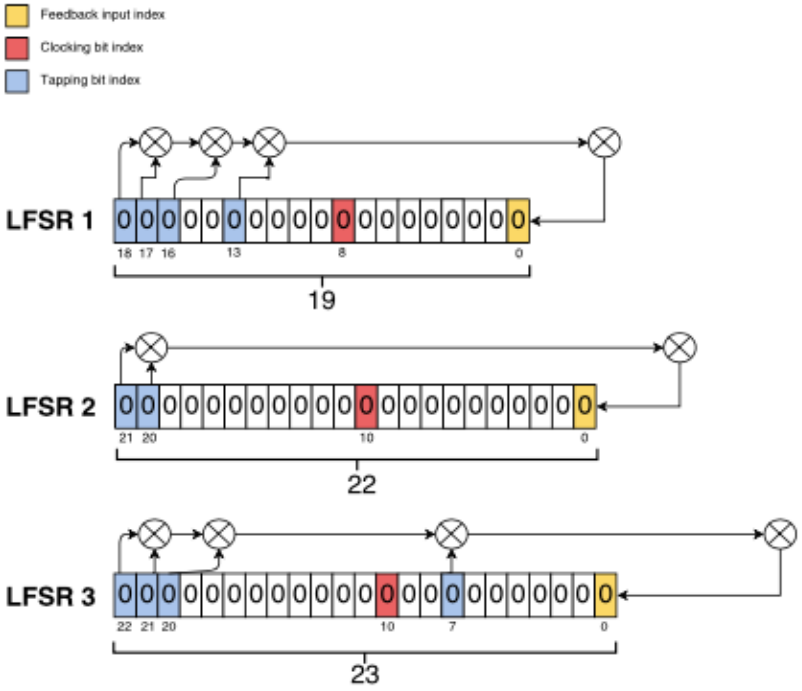


- MSB=Most Significant Bit
- LSB=Least Significant Bit

A5/1: ARCHITECTURE

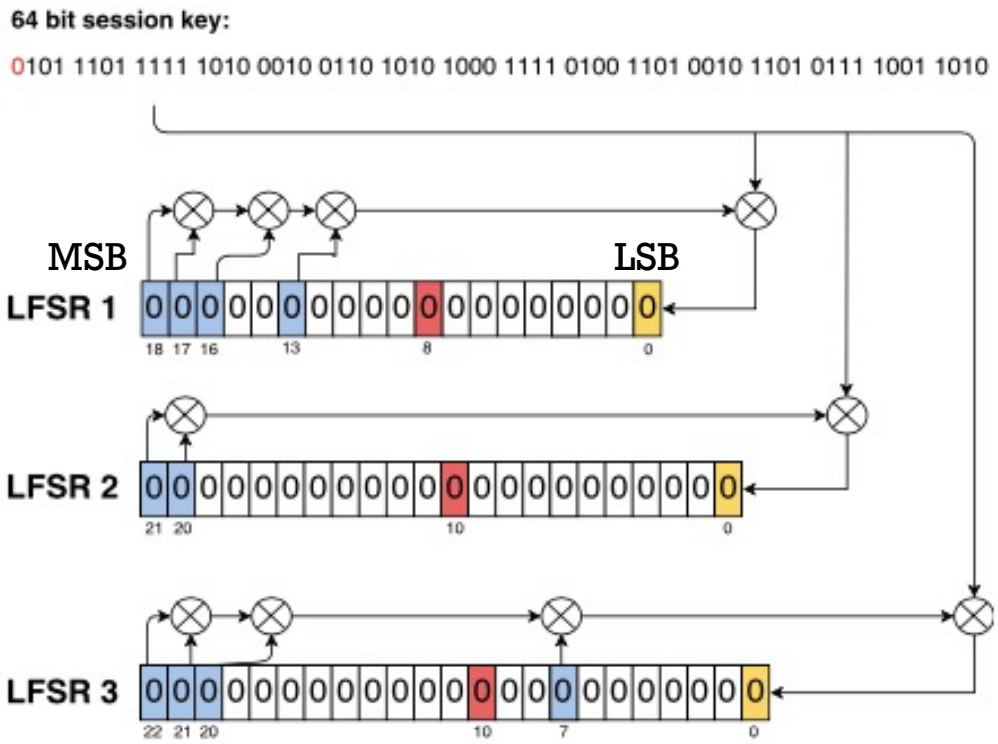
- When a register is clocked, its tapped bits are XORed and the result is stored in the registers LSB
- The registers MSB is shifted out of the register, and its value is forgotten

A5/1: STEP 1, INIZIALIZATION



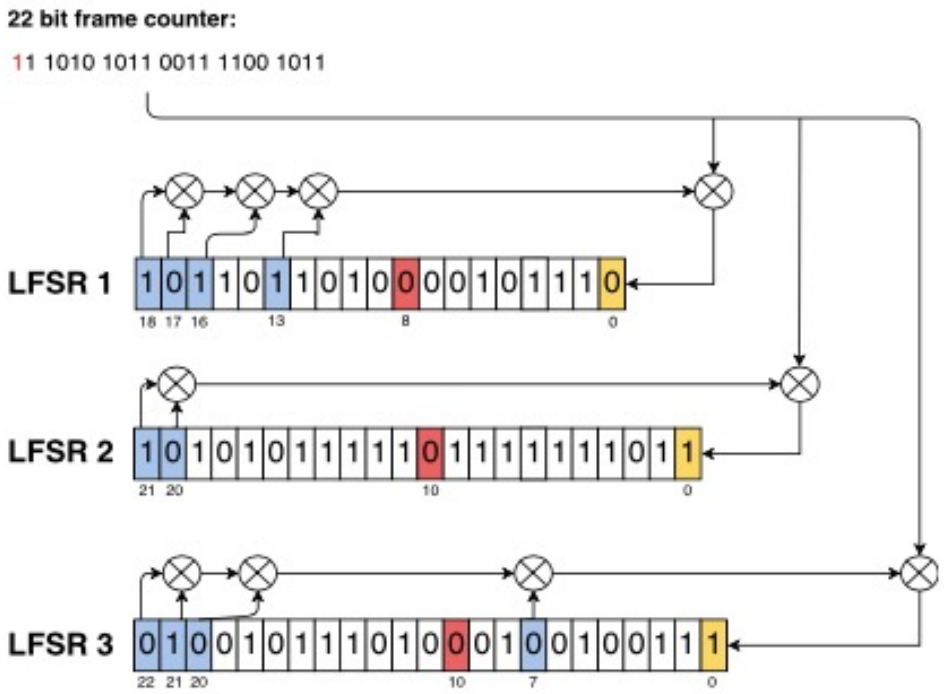
A5/1: STEP 2, CLOCKING WITH SESSION KEY

- After registers are initialized with 0s…
- … they are clocked 64 times
  - one time for each bit in the session key
- The bits of the 64 bit session key are consecutively XORed in parallel with the feedback of the register, and the result is fed into the LSB of the respective register
- First value of the session key is highlighted in red and is ready to be fed into the circuit
- Each cycle will result in a new (unique) state
- This step is finished when the last bit of the session key is fed into the circuit



A5/1: STEP 3, CLOCKING WITH FRAME COUNTER

- At the end of step 2, registers do not more hold only 0s
- This step is similar to previous
- Main difference: instead of session key, a (public) frame counter (Initialization Vector) is used
- The length of the frame counter is 22 bits
- The registers are thus clocked 22 times to inject all bits in the frame counter



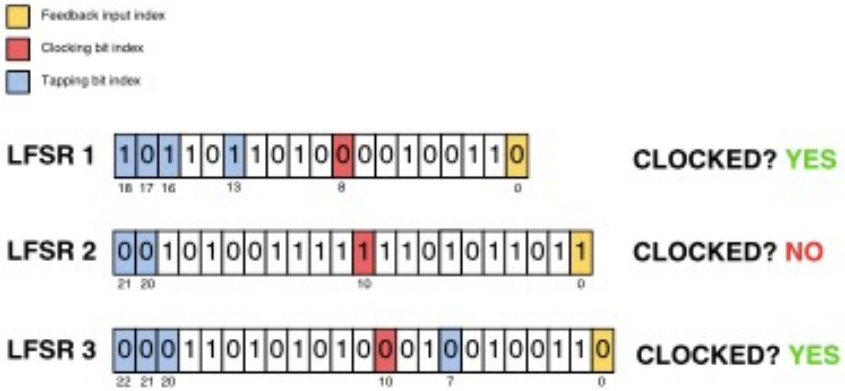
A5/1: STEP 4, CLOCKING WITH MAJORITY

- The registers are clocked 100 times with **irregular clocking**
  - This is where the clock bits come into play
- Irregular clocking follows the majority rule, a decision rule that selects alternatives that have a majority
  - The majority bit is determined by the **clocking bits** of the registers
    - LFSR 1 clocking bit: 8
    - LFSR 2 clocking bit: 10
    - LFSR 3 clocking bit: 10
  - **If a clocking bit of a register is equal to the majority bit, the register is clocked**
  - **Otherwise, the register is left unchanged**

EXAMPLES OF MAJORITY

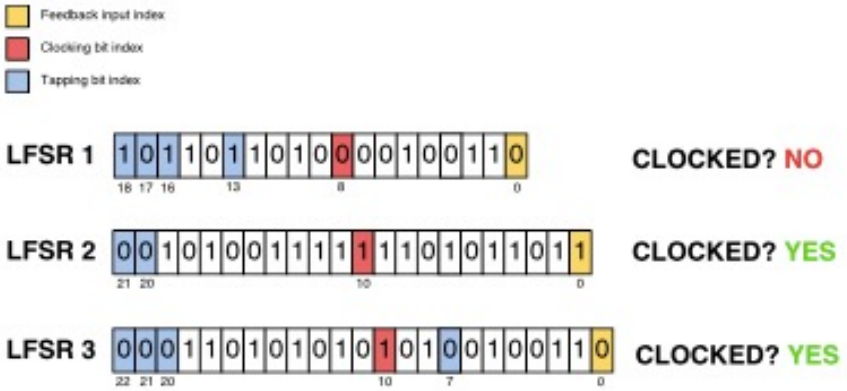
CB of $R1$: $R1[8]$	0	0	0	0	1	1	1	1
CB of $R2$: $R2[10]$	0	0	1	1	0	0	1	1
CB of $R3$: $R3[10]$	0	1	0	1	0	1	0	1
R1 clocked?	√	√	√	–	–	√	√	√
R2 clocked?	√	√	–	√	√	–	√	√
R3 clocked?	√	–	√	√	√	√	–	√

CB = clocking bit



Maj(LFSR1[8],LFSR2[10],LFSR3[10])=Maj(0,1,0)=0

- LFSR1 is clocked as LFSR1[8]=0
- LFSR2 is not clocked as LFSR2[10] != 0
- LFSR3 is clocked as LFSR3[10]=0

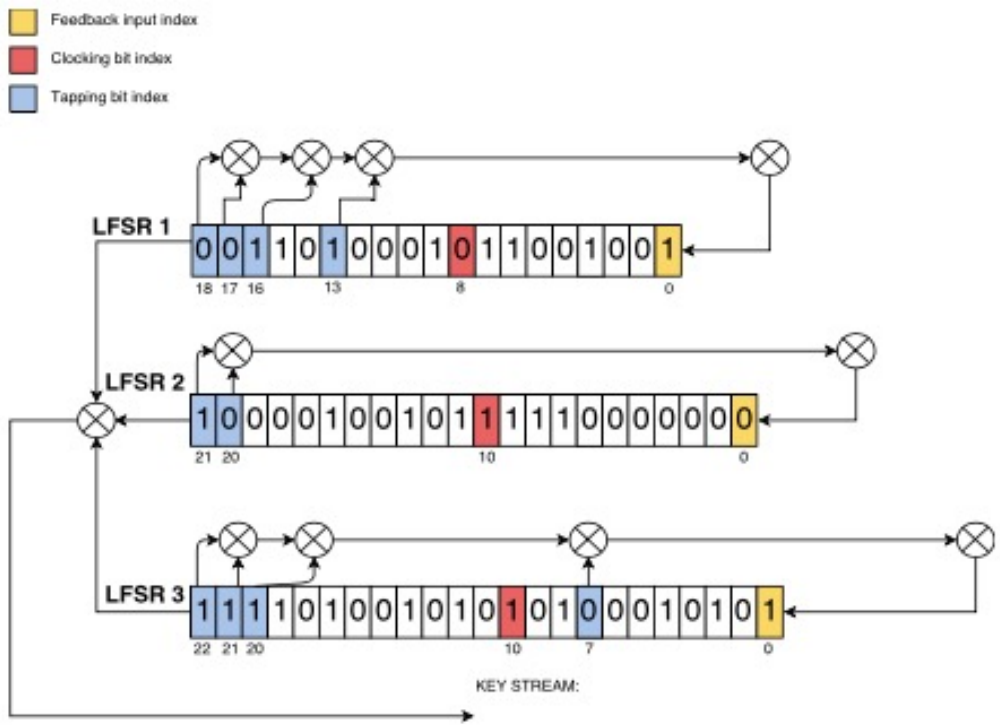


Maj(LFSR1[8],LFSR2[10],LFSR3[10])=Maj(0,1,1)=1

- LFSR1 is not clocked as LFSR1[8] != 1
- LFSR2 is clocked as LFSR2[10] = 1
- LFSR3 is clocked as LFSR3[10]=1

A5/1: STEP 5, PRODUCING KEY STREAM

- The registers are **clocked 228 times with irregular clocking**
- For each cycle, the MSB of each register is used for a final computation
  - **Each register's MSB is XORed with one-another, and the result is added to the key stream**
- Since each cycle generates a key stream bit, and we cycle our circuit 228 times, the output key stream will consist of 228 bits



A5/1: STEP 6, CREATING THE CIPHERTEXT

- Now that we have a pseudo random generated key stream, we can encrypt plain texts
- Only 114 bits of the key stream can be used to encrypt our data, as 114 bits will be used to decrypt the incoming message
- To encrypt data, we take a 114 bit chunk of plain text and XOR it bitwise with the 114 bits of key stream dedicated for encryption
- This simple XOR between plain text and key stream creates the cipher text, which we can send **securely** over the GSM network

24

A5/1 IN 3 SLIDES

A5/1: KEY LOADING (STEPS 1-3)

- **Seed**: a (secret) vector *K* of 64 bits (session key)
- **Initialization Vector**: a public vector *IV* of 22 bits (frame counter)
- From *K* and *IV* we get the initial state (namely a vector of 64 bits) using a **key-loading function** *kl(K,IV)*

- The function *kl* is defined by iteration: initially the registers contain all zeros, the bits in *K* are injected by xor-ing them with other bits, and similarly for the bits of *IV*

A5/1: WARM UP (STEP 4)

- After key loading, the registers R1, R2, and R3 are **irregularly clocked** 100 times, without producing output
- The irregular clock works as follows:
  - in each step, the **clock bits**
  - Each one among R1, R2, and R3 is updated if the clock bits agree with the majority of the clock bits
- In this way, it is possible to show that each register clocks with a probability of 3/4
- At this point, the initialization of the registers is complete and the stream cipher is ready to output the key-stream (denoted with Z)

A5/1: UPDATE & OUTPUT (STEPS 5 AND 6)

- The key-stream is composed of 228 bits that are obtained in 228 steps
- At each clock tick, the functions are defined as follows
  - **Update**
    - The register R1 is updated if its clock bit agrees with the majority of the others
    - The register R2 is updated if its clock bit agrees with the majority of the others
    - The register R3 is updated if its clock bit agrees with the majority of the others
  - **Output**: the xor of the output of the three registers R1, R2, R3 is the output bit
  - After 228 bits of output, the key-loading phase is executed again

A5/1: SOME ATTACKS IN BRIEF (1)

- In GSM phones, the session keys are produced with another algorithm, which **might depend on the operator**

- Marc Briceno, Ian Goldberg and David Wagner published in 1999 the first complete description of A5/1 and claimed that all the implementations they checked used 54-bit session keys
  - I.e. 64-bit keys with 10 fixed bits set to 0s
- Although other operators could decide otherwise, it is probable that this convention will be maintained by operators in the name of backward compatibility

If you want to have a look at the code implementing A5/1, please, see at https://cryptome.org/jya/a51-pi.htm

From 2^64 to 2^54 key space… a reduction of more than 10^3

A5/1: SOME ATTACKS IN BRIEF (2)

- Anderson and Roe proposed an attack based on guessing the 41 bits in the shorter R1 and R2 registers, and deriving the 23 bits of the R3 register from the output
  - If we assume that a PC can test ten million guesses per second, Anderson and Roe's method would still need more than a month to find one key

- Golic designed a complex attack, where each step is based on the solution of a system of linear equations
  - Even though the attack requires fewer steps, each step is more complex, and does not run faster than previous attacks on a regular computer
  - J. Golic. *Cryptanalysis of alleged A5 stream cipher*. Proceedings of EUROCRYPT'97, LNCS 1233, pp.239-255,1999.

- For more an almost real-time attack, see https://cryptome.org/a51-bsw.htm

30

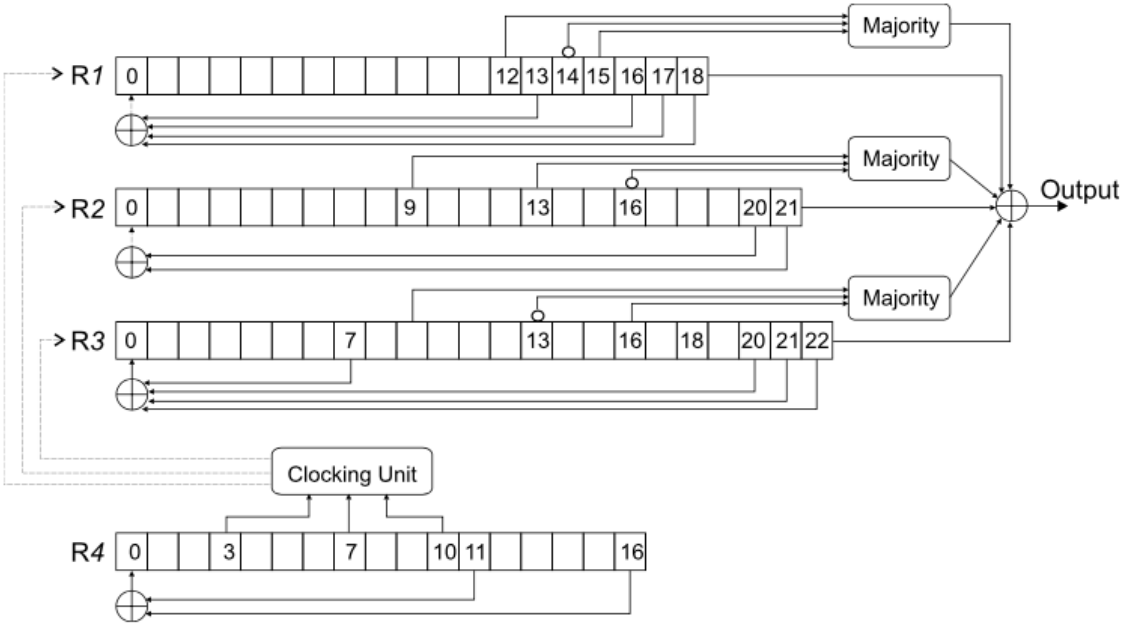
A5/2

A5/2: MAIN FEATURES

- year: 1989
- keystream length: 228 bits
- Number of LFSRs: 4
- update of registers: majority function on the control bits in R4
- Registers
  - lengths: 19, 22, 23, 17 (total: 81 bits of state)
  - R1, R2, R3, R4 are maximum-length LFSRs as the polynomials are primitive

Register	Feedback Polynomial	Taps	Clock
R_1	$x^{19} + x^{18} + x^{15} + x^{14} + 1$	18,17,14,13	-
R_2	$x^{22} + x^{21} + 1$	21,20,	-
R_3	$x^{23} + x^{22} + x^{21} + x^{8} + 1$	22,21,20,7	-
R_4	$x^{17} + x^{12} + 1$	16,11	3,7,10

A5/2: ARCHITECTURE



- Main differences between A5/1 and A5/2
  - A fourth register
  - A slightly different initialization phase
  - The input bits for the Clocking Unit are taken from R4 only
  - The output is an XOR of
  1. the MSB of R1, R2 and R3
  2. the majority of 3 bits of each register with one of these 3 bits negated:

Register	MSB	Majority bits	Negated majority bit
R_1	$R_1[18]$	$R_1[12]$, $R_1[15]$	$R_1[14]$
R_2	$R_2[21]$	$R_2[9]$, $R_2[13]$	$R_2[16]$
R_3	$R_3[22]$	$R_3[16]$, $R_3[18]$	$R_3[13]$

A5/2: FIRST HALF STEPS

- All registers are set to 0s
- The registers are clocked for 64 cycles:
  - In each cycle i ($0 \leq i \leq 63$) the bit Kc[i] is XOR'ed with the LSB of the register and stored in the LSB of the same register
- The registers are clocked for 22 cycles:
  - In each cycle i ($0 \leq i \leq 21$) the bit Fn[i] is XOR'ed with the LSB of the register and stored in the LSB of the same register
- Set bits R1[15], R2[16], R3[18], R4[10] to 1

A5/2: SECOND HALF STEPS

- The next 99 cycles are run to diffuse Kc and Fn into the registers, discarding the output
  - Irregular clocking is applied: Whether a register is clocked or not is determined during each cycle by the Clocking Unit calculating the majority of all 3 clocking bits R4[3], R4[7], R4[10] - if the majority matches the clocking bit, the corresponding register is clocked
  - Register R4 is always clocked last in every cycle
- The next 228 cycles are again carried out with the same irregular clocking as in the previous step
  Each cycle i ($0 \leq i \leq 227$) the MSBs of all 3 registers are XOR'ed and the result is used as bit i of the output

$R_4[3]$	$R_4[7]$	$R_4[10]$	Majority	R_1	R_2	R_3
0	0	0	0	clock	clock	clock
1	0	0	0	clock		clock
0	1	0	0	clock	clock	
1	1	0	1		clock	clock
0	0	1	0		clock	clock
1	0	1	1	clock	clock	
0	1	1	1	clock		clock
1	1	1	1	clock	clock	clock

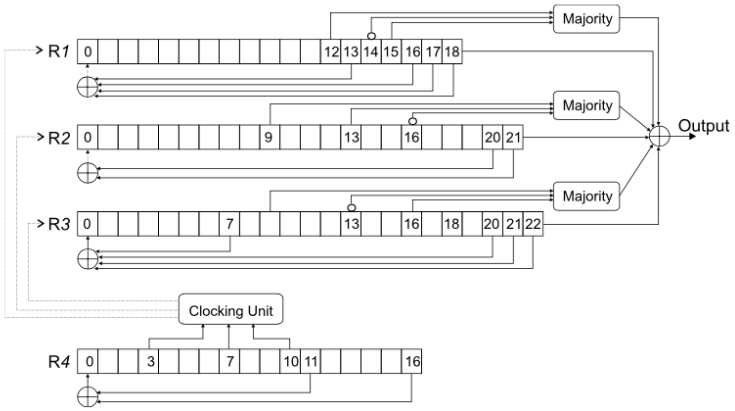
A5/2 IN 2 SLIDES

A5/2: KEY LOADING & WARM UP

- **Seed**: a (secret) vector *K* of 64 bits
- **Initialization Vector**: a public vector *IV* of 22 bits
- From *K* and *IV* we get the initial state (namely a vector of 64 bits) using a **key-loading function** *kl(K,IV)*
- The function *kl* is defined by iteration by using a similar approach to that of the A5/1 cipher

- Warm-up: defined in a way similar to that of A5/1

A5/2: UPDATE & OUTPUT



At each clock tick, the functions are defined as follows

- **Update**
  - R_1 is updated if $R_4[10] = maj(R_4[10], R_4[3], R_4[7])$
  - R_2 is updated if $R_4[3] = maj(R_4[10], R_4[3], R_4[7])$
  - R_3 is updated if $R_4[7] = maj(R_4[10], R_4[3], R_4[7])$
  - R_4 is always updated

- **Output**
  - A majority function is applied as non-linear filtering function to each of the first three registers:
    - for R_1 we compute $maj(R_1[12], 1+R_1[14], R_1[15]) = a_1$
    - for R_2 we compute $maj(R_2[9], R_2[13], R_2[16]+1) = a_2$
    - for R_3 we compute $maj(R_3[13]+1, R_3[16], R_3[18]) = a_3$

$$z = R_1[18] + R_2[21] + R_3[22] + a_1 + a_2 + a_3$$

A5/2: CRYPTANALYSIS

- In 1999, Ian Goldberg and David A. Wagner cryptanalyzed A5/2 in the same month it was published, and showed that it was **extremely weak**
  - Known plaintext attack
  - Low end equipment can probably break it in real time
- Known plaintext attack
  - Difference of two given (plaintext) frames which are roughly 6 seconds apart
  - The average computation cost is about 216 operations of 114-bit vectors
- Since July 1, 2006, the GSM Association mandated that GSM Mobile Phones should not support the A5/2 Cipher any longer

39

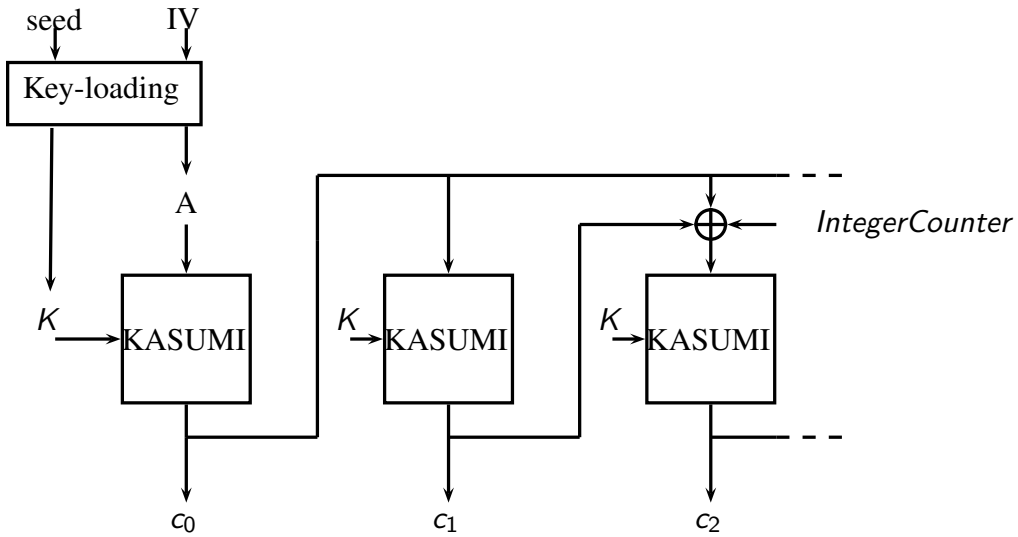
A5/3

A5/3: OVERVIEW

- A5/3 is the last stream cipher of the A5 family and provides users with a higher level of security than both A5/1 and A5/2

- Used in GSM, UMTS (f8-UEA1, f9-UIA1) and GPRS (GEA3) communications systems

- It is based on the **block cipher** KASUMI
  - Key space: 128 bits
  - Message space: 64 bits
    - Recall that there is one encryption function per key



A5/3: ARCHITECTURE

- We will see block ciphers in the next lecture

REMARKS ON THE A5 FAMILY (1)

The generated key (*Kc*) is the main flaw of the design:

- Maximal exhaustive search complexity is (only) 2^(64)
- Kc generated only once after the cell phone registers with the network and stays active for all communication, until the telco requests a new one or the cell phone deregisters
- *Kc* is artificially shortened in deployed systems when zeroing 10 bits
- Encryption is applied after error correction
  - Barkan, Biham and Keller proposed a very practical attack requiring only a few encrypted frames from a conversation to discover *Kc*
  - Started with known-plaintext attack based and then converted to a ciphertext-only attack by taking advantage of the error correction codes implemented in GSM data
  - http://www.cs.technion.ac.il/users/wwwb/cgi-bin/tr-get.cgi/2003/CS/CS-2003-05.pdf

REMARKS ON THE A5 FAMILY (2)

- **A5/2** is **extremely weak** and it can be broken in real time with inexpensive equipment; it is therefore **no longer supported** by new mobile phones

- **A5/1** is affected by a **number of serious weaknesses**, and its use is strongly **discouraged**, since there are practical attacks that can break the cipher

- **A5/3** is the **common standard for the new generation of mobile** and it is **considered secure**, even though there do exist practical attacks to KASUMI that suggest some significant weaknesses of the cipher
  - O. Dunkelman and N. Keller and A. Shamir. *A Practical-Time Attack on the A5/3 Cryptosystem Used in Third Generation GSM Telephony.* Cryptology ePrint Archive, Report 2010/013, 2010, http://eprint.iacr.org/

ON THE PRACTICALITY OF ATTACKING GSM COMMUNICATIONS

- In theory, the attacks are relatively simple
- In practice, a considerable amount of hardware is necessary to actually intercept GSM communications
- The hardware must at least consist of a radio receiver device which is capable of receiving and decoding digital data that is exchanged over-the-air by using one of the attacks reported above
- Hypothetically a simple GSM mobile phone already has all these capabilities (except the decrypting of an unknown A5 stream), so it might be possible to use such a phone for eavesdropping
- Nevertheless a huge amount of know-how, time and money is needed

45

E0

BLUETOOTH

- Bluetooth is a wireless technology standard invented by Ericsson in 1994 for exchanging data over short distances using short-wavelength UHF **radio** waves (Range: 2.4 to 2.485 GHz) from fixed and mobile devices
  - https://www.bluetooth.com/
- It allows devices to communicate with each other by splitting data into packets that are exchanged through one of 79 designated channels (each of which have 1 MHz in bandwidth)
- It was originally designed for continuous, streaming data applications, i.e. it is possible to exchange a lot of data at a close range: cellular phones, wireless headsets, printers, cars, and turnstiles
- The standard offers methods for generating keys, authenticating users, and **encrypting** data

BLUETOOTH LOW ENERGY (BLE)

- BLE was introduced in 2011 as Bluetooth 4.0
- Main difference between BLE and Bluetooth: power consumption
- In Machine 2 Machine (M2M) applications such as those supported by the Internet of Things (IoT), the use of BLE allows applications run on a small battery for 4/5 years!
- Although this is not ideal for talking on the phone, it is vital for **applications that only need to exchange small amounts of data periodically**
- BLE operates in the same radio frequency band of Bluetooth but remains in sleep mode constantly except for when a connection is initiated
- The actual connection times are only a few milliseconds
  - Bluetooth would take around 100 milliseconds
  - Data rates are very high at 1 Mb/s

BLE's M2M/IoT Applications

- Blood pressure monitors
- Fibit-like devices
- Industrial monitoring sensors
- Geography-based, targeted promotions (iBeacon)
- Public transportation apps

BLE uses the AES cipher with 128-bit key length to provide data encryption and integrity over the wireless link

E0

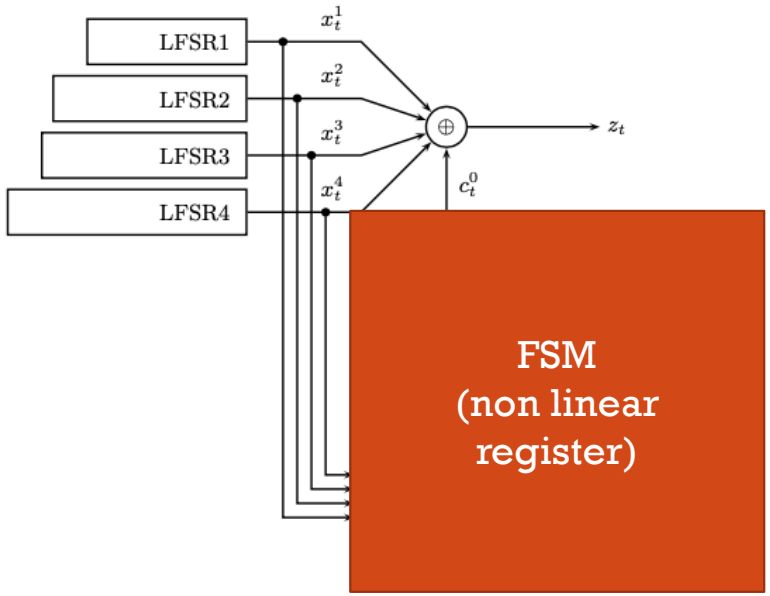
- year: 1999
- used in Bluetooth standard
- number of LFSRs: 4 Linear registers:
- Linear registers
  - lengths: 25, 31, 33, 39 (total for the linear part: 128 bits)
- number of non-linear registers: 1
- Non-linear registers
  - length of the non-linear register: 4 bit (total: 132 bits)
- update functions: depend on the non-linear register

These are all primitive

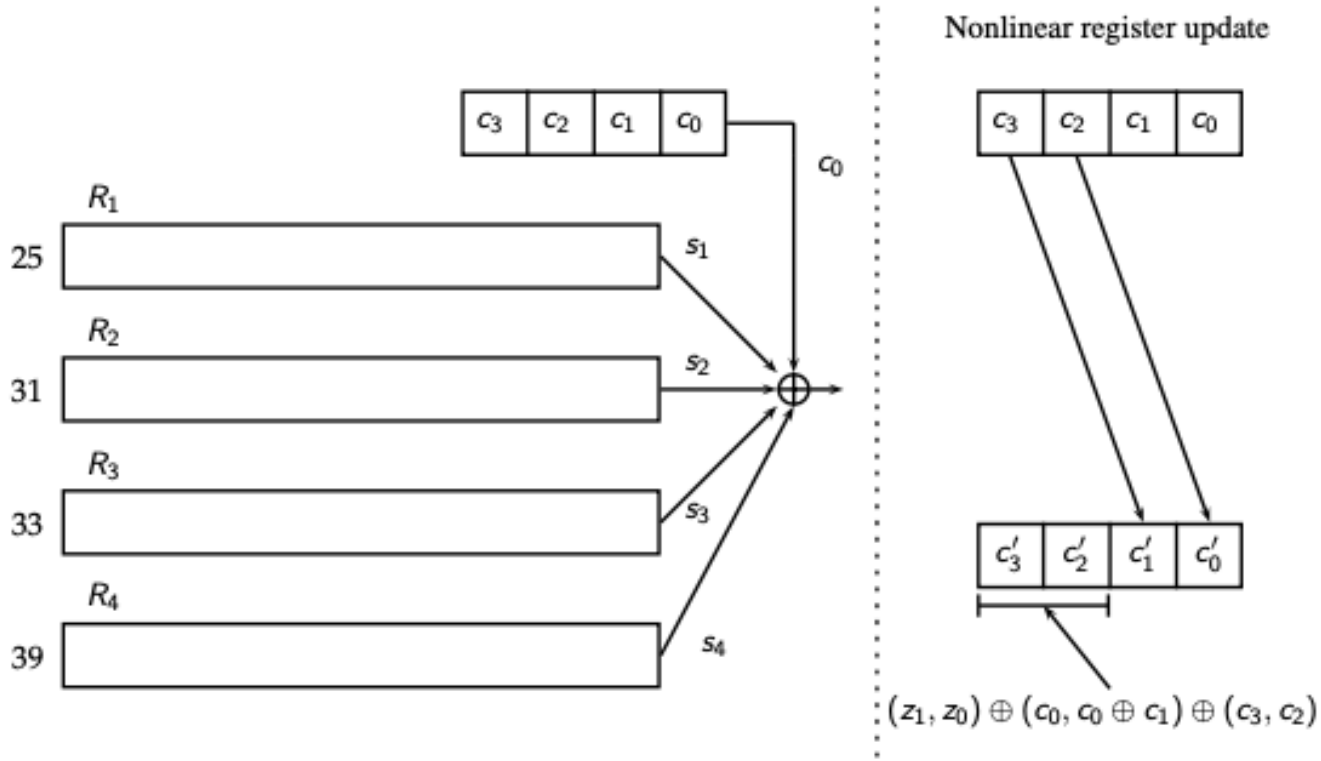
Register	Feedback Polynomial	Taps
R_1	$x^{25} + x^{20} + x^{12} + x^8 + 1$	$0, 8, 12, 20$
R_2	$x^{31} + x^{24} + x^{16} + x^{12} + 1$	$0, 12, 16, 24$
R_3	$x^{33} + x^{28} + x^{24} + x^4 + 1$	$0, 4, 24, 28$
R_4	$x^{39} + x^{36} + x^{28} + x^4 + 1$	$0, 4, 28, 36$

E0: OVERVIEW

- The key stream generator consists of 4 LFSRs whose outputs are combined by a simple finite state machine
- The LFSRs are all of different length and contain a total of 128 bits
- All 4 feedback polynomials are primitive, the LFSRs will generate maximum-length sequences
- The state of the FSM is determined by 4 bits, which are stored in a pair of 2-bit delay elements
- Encryption is performed by taking the xor of the output sequence and the bit sequence of a data packet
- The maximum size of such a packet is limited to 2745 bits.
  - When the transmission of this packet is completed, the cipher is reinitialised



E0: ARCHITECTURE



FSM

SOME PRELIMINARY OBSERVATIONS

- We will not use + for xor
- Instead, we use + as the usual addition over integers and $\oplus$ for the xor operation over bits
- A bit x can be also written as an integer, the value of which is 1 or 0. In the same way, the integers 0 and 1 can be seen as bits
  - Example: $1+1=2$ and $1 \oplus 1=0$
- Given two bits x and y, we can compute the integer $2x + y$ which will be in the range $[0,3]$
- We write the integers $0, 1, 2,$ and 3 as bit vectors of length 2: $00, 01, 10$ and 11

E0: OUTPUT & UPDATE (1)

- Output
  - To obtain a bit of keystream we XOR the output of each linear register and the output of the non-linear register: $x = s_1 \oplus s_2 \oplus s_3 \oplus s_4 \oplus c_0$

    where $s1, s2, s3$ and $s4$ indicate the output bits of the 4 LFSRs

- Update
  - The update of the 4 LFSRs is regular (without control bits)
  - To describe the update of the non-linear register, we define $z = \left\lfloor \frac{s_1 + s_2 + s_3 + s_4 + 2c_3 + c_2}{2} \right\rfloor$

    noting that $s_1 + s_2 + s_3 + s_4 + 2c_3 + c_2$ is an integer number (recall that + is the sum of integers)
  - Hence $0 \leq z \leq floor(7/2) = 3$ and we consider its binary representation (z_1, z_0)

E0: OUTPUT & UPDATE (2)

- The update of the non-linear register is given by

$$(c_3, c_2, c_1, c_0) \longmapsto (c'_3, c'_2, c'_1, c'_0)$$

where

- $c'_1 = c_3$ and $c'_0 = c_2$, i.e. c_1 and c_2 are shifted of two places to the right
- $(c'_3, c'_2) = (z_1, z_0) \oplus (c_0, c_0 \oplus c_1) \oplus (c_3, c_2)$.

Example

Let us consider the state of the non-linear register

$$(c_3, c_2, c_1, c_0) = (1, 0, 1, 1),$$

and let $(s_1, s_2, s_3, s_4) = (1, 1, 0, 0)$ *be the output of the four linear registers.*

To update the non-linear register we first compute z*:*

$$z = \left\lfloor \frac{s_1 + s_2 + s_3 + s_4 + 2c_3 + c_2}{2} \right\rfloor = \left\lfloor \frac{1 + 1 + 0 + 0 + 2 \cdot 1 + 0}{2} \right\rfloor = 2$$

Hence the binary expression of z *is* $(z_1, z_0) = (1, 0)$.

Example (continued)

- $(c_3, c_2, c_1, c_0) = (1, 0, 1, 1)$
- $(z_1, z_0) = (1, 0)$

We recall how the non-linear register is updated:

- $c'_1 = c_3$ *and* $c'_0 = c_2$
- $(c'_3, c'_2) = (z_1, z_0) \oplus (c_0, c_0 \oplus c_1) \oplus (c_3, c_2)$.

Hence

$$(c'_3, c'_2) = (1, 0) \oplus (1, 1 \oplus 1) \oplus (1, 0) = (1, 0)$$

and therefore the updated non-linear register is

$$(c'_3, c'_2, c'_1, c'_0) = (1, 0, 1, 0)$$

REMARK ON NON-LINEARITY

- Note that the non-linear part is introduced in the definition of *z*, because

  summing vectors of GF(2^n) as integers is a non-linear operation on the vector space GF(2^n)

- This is a good technique to construct stream ciphers, since it allows to reach high non-linearity using easy and fast computations

ATTACKS ON E0 (1)

- Known plaintext attacks
  - An attacker is able to obtain a certain amount of decrypted text in one way or another
  - Goal: use availabe information to recover other (unknown) parts of the plaintext

- In the case of E0 (that is an additive stream cipher), the attack amounts to finding a way to predict the entire key stream given a limited number of key stream bits

- Two possible approaches
  - **Correlation** attacks
  - **Guess and determine** attacks

ATTACKS ON E0 (2)

- **Correlation**
  - Recover parts of the initial state of the generator by exploiting specific correlation properties of the FSM
  - Cannot be applied to the E0 algorithm, as it assumes sequences of consecutive key stream bits which are considerably longer than the maximum packet size
- **Guess and determine**
  - some parts of the initial state are guessed first and the observed key sequence is used to derive the remaining parts in a deterministic way afterwards
  - Example
    - The contents of a single LFSR can directly be derived from the contents of the three other LFSRs, the 4 bits of the FSM, and some known key stream
    - By guessing the initial state of the FSM and the contents of the three shortest LFSRs (4 + 25 + 31 + 33 = 93 bits), the attack succeeds in recovering the entire internal state in 292 steps

59

RC4

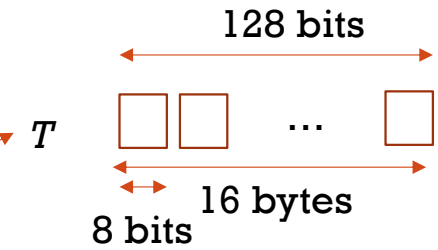
RC4: HISTORY

- It was designed by Ron Rivest of RSA Security in 1987
  - RC4 = Rivest Cipher 4
- Initially a trade secret, but in September 1994 a description of it was leaked
  - The name RC4 is trademarked
  - RSA Security has never officially released the algorithm
  - Rivest has confirmed the history of RC4 and its code in 2014
- RC4 became part of some commonly used encryption protocols and standard
  - WEP in 1997 and WPA in 2003/2004 for wireless cards
  - SSL in 1995 and its successor TLS in 1999
- The main factors in RC4's success over such a wide range of applications have been its **speed and simplicity**: efficient implementations were easy to develop

RC4: OVERVIEW

- RC4 generates a pseudorandom stream of bytes, the key stream
- To generate the keystream, the cipher makes use of a secret internal state which consists of two parts:
  - A permutation of all 256 possible bytes (denoted *S*)
  - Two 8-bit index-pointers (denoted *i* and *j*)
- The permutation is initialized with a **variable length key, typically between 40 and 2048 bits**, using the so-called **key-scheduling algorithm** (**KSA**)
- Once this has been completed, the stream of bits is generated using the pseudo-random generation algorithm (PRGA)
- While many stream ciphers are based on LFSRs (efficient in hardware but less so in software), the **design of RC4 avoids the use of LFSRs** and is **ideal for software implementation, as it requires only byte manipulations**

RC4: SETUP



- Fundamental to the RC4 algorithm is a 256 element array of 8-bit integers
  - It is called the state vector and denoted *S*
- The state vector is initialized with the encryption key
  - The initialization steps are as follows:

1. *S* is initialized with entries from 0 to 255 in the ascending order
2. *S* is further initialized with the help of a temporary 256-element vector denoted *T* that also holds 256 integers. The vector *T* is initialized with the encryption key as follows:
   a) Let *K* be the encryption key represented as a vector 8-bit integers of size 16 (in case of a 128-bit key), i.e. *K* stores 16 non-negative integers whose values will be between 0 and 255
   b) Initialize the 256-element vector *T* by placing in it as many repetitions of the key as necessary until T is full
3. Use the 256-element vector *T* to produce the initial **permutation** of *S* as follows:

```
j  =  0
for i = 0 to 255
    j  =  ( j + S[i] + T[i] )  mod  256
    SWAP  S[i], S[j]
```

Key Scheduling Algorithm (KSA)

GENERATING THE PSEUDORANDOM BYTE STREAM

- **Idea**: as each byte of the plaintext becomes available, it is XORed with a byte of the pseudorandom byte stream
- **Procedure**

```
i, j  =  0
while ( true )
    i  =  ( i + 1 )  mod  256
    j  =  ( j + S[i] )  mod  256
    SWAP S[i], S[j]
    k  =  ( S[i]  +  S[j] )  mod  256
    output S[k]
```

- Note how the state vector S changes continuously because of swapping at each pass through the body of the loop
- The **state** of the generator **changes dynamically** as the the numbers are being generated

RC4: REMARKS

- Theoretical analysis shows that for a **128 bit key length**, the **period** of the pseudorandom sequence of bytes is likely to be **greater than 10^(100)**
- As all operations are at the byte level, the cipher possesses **fast software implementation**
  - For this reason, RC4 was the software stream cipher of choice for several years
  - More recently though, **RC4 was shown to be vulnerable to attacks** especially if the beginning portion of the output pseudorandom byte stream is not discarded
  - As a consequence, the use of RC4 in the SSL/TLS protocol is now prohibited (since 2015)
- WiFi security started with RC4 in the WEP protocol
- After it was discovered that the encryption key used in WEP could be acquired by an adversary in almost no time, WiFi security has now moved on to the WPA2 protocol that uses AES for encryption

RC4: SOME SECURITY CONSIDERATIONS

- Unlike modern stream ciphers, **RC4 does not take a separate nonce alongside the key**
  - If a single long-term key is to be used to securely encrypt multiple streams, the protocol must specify how to combine the nonce and the long-term key to generate the stream key for RC4
  - One approach to addressing this is to generate a "fresh" RC4 key by hashing a long-term key with a nonce
  - Unfortunately, many applications that use RC4 simply concatenate key and nonce; this gives rise to related key attacks that are famous for breaking the WEP standard
- Because RC4 is a stream cipher, it is **more malleable** than block ciphers
  - If not used together with other techniques (e.g., message authentication codes), then encryption is vulnerable to bit-flipping attacks

FINAL REMARKS ON STREAM CIPHERS

- Stream ciphers require a shared secret, a **seed**, which should be transmitted via a **secure channel**
  - Once Alice and Bob have the seed, they can exchange via an insecure channel other paramaters, like the IV and start encrypting/decrypting
- The use of **stream ciphers guarantees confidentiality**
  - Eve will not be able to understand what Alice and Bob are telling each other
- However, the **stream cipher** in itself **does not guarantee that the seed has been exchanged securely**, this has to be done in other ways
- On the other hand, the attacker might tamper with the ciphertext and Alice/Bob will not understand immediately that the messages have been corrupted
  - In other words, **stream ciphers provide neither authentication nor integrity**